LAB REPORT 9

Deploying Custom Socket Applications on Mininet-WiFi Nodes

Student Name	Hadis Rodpradit
Student ID	6630613039
Program	Digital Engineering International Program, College of Computing, PSU Phuket
Course	977-337 Network Programming and Simulation
Week	9
Environment	Ubuntu 24.04 Linux VM, Mininet-WiFi Python API (`mn_wifi`), Open vSwitch AP, Python 3, matplotlib 3.11.1
Source files	`lab09_app_integration/`: `echo_server.py`, `echo_bench_client.py`, `app_topo.py`, `plot_rtt.py`, `rtt_near.json`, `rtt_far.json`, `rtt_plot.png`

1. Objective

This lab deploys the TCP echo server written earlier in the course (Lab 2) inside a Mininet-WiFi station without changing a single line, and drives it from a new non-interactive benchmark client that sends 50 messages of 64 bytes and times each round trip. A single Python script builds the topology, starts the server on sta1, runs the benchmark from sta2 at 5 m from the AP, moves sta2 to 28 m (near the edge of `range=30`) and runs it again. The two RTT series are then plotted and compared with the iperf3/ping measurements from Lab 8.

2. Topology and Measurement Setup

Node / setting	Value	Role
ap1	position 0,50,0 · SSID netprog-ap · mode g · ch 1 · `range=30`	single access point, `failMode='standalone'`
sta1	position 0,50,0 · 10.0.0.1	runs `echo_server.py` (TCP port 5000)
sta2	5,50,0 (near) → 28,50,0 (far) · 10.0.0.2	runs `echo_bench_client.py`
Benchmark	1 TCP connection, 50 messages × 64 bytes, back-to-back	RTT = time from `sendall()` until the full echo is received
Distance vs. range	5 m = 17 % · 28 m = 93 % of range	28 m is just inside ap1's coverage

Expected result from Lab 8: ping RTT was 0.69 ms at 5 m and 1.97 ms at 30 m and rose roughly linearly, so at 28 m the network part of the RTT should be about 0.69 + (1.97 − 0.69) × 23 / 25 ≈ **1.87 ms**, plus a little extra for TCP and Python at the application level.

3. Final Scripts and Changes Needed in This Environment

`echo_server.py` was copied from my Lab 2 folder (named `lab03_tcp_echo` in my repository) and used **unmodified**: it already binds to `0.0.0.0` on port 5000 (Figure 1). The other scripts follow the lab sheet, with these additions:

#	Problem / risk	Cause	Fix
1	Traffic through the OVS access point is dropped when no controller runs (known from Lab 7)	A script using `Mininet_wifi()` starts no controller	`failMode='standalone'` on ap1 in both topologies
2	The sheet's multi-line `python3 -c "..."` for Part B cannot be typed in the CLI	The Mininet CLI executes each line as a separate command	Same client written on one line, separated by semicolons (Figure 2)
3	One `recv(64)` may return fewer than 64 bytes	TCP is a byte stream, not a message protocol	Loop until all 64 bytes of the echo have arrived before stopping the timer
4	Server output could mix into later `cmd()` output	A background process shares the node's shell	Redirect it to `/tmp/echo_server.log`; `pkill` the server before `net.stop()`

I also added a standard deviation (`std`) and the message count (`n`) to the summary line to quantify how variable each run is. Final `echo_bench_client.py` (Part C):

```
import socket
import sys
import time
import json

PORT = 5000  # must match the port used by echo_server.py (Lab 2)


def run_benchmark(server_ip, port=PORT, count=50, msg_size=64):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.connect((server_ip, port))
    payload = b"x" * msg_size
    rtts_ms = []
    for i in range(count):
        # TODO 1
        start = time.perf_counter()
        sock.sendall(payload)
        # TCP is a byte stream: keep reading until the whole echo is back
        received = 0
        while received < msg_size:
            chunk = sock.recv(msg_size - received)
            if not chunk:
                raise ConnectionError("server closed the connection")
            received += len(chunk)
        # TODO 2
        rtts_ms.append((time.perf_counter() - start) * 1000)
```

```
    sock.close()
    return rtts_ms


if __name__ == "__main__":
    server_ip = sys.argv[1]
    label = sys.argv[2] if len(sys.argv) > 2 else "run"
    rtts_ms = run_benchmark(server_ip)
    avg = sum(rtts_ms) / len(rtts_ms)
    std = (sum((r - avg) ** 2 for r in rtts_ms) / len(rtts_ms)) ** 0.5
    print(f"{label}: min={min(rtts_ms):.3f}ms avg={avg:.3f}ms max={max(rtts_ms):.3f}ms "
          f"std={std:.3f}ms n={len(rtts_ms)}")
    with open(f"rtt_{label}.json", "w") as f:
        json.dump(rtts_ms, f)
```

Final `app_topo.py` (Part D with the Part E extension — the version pushed to the repository):

```
from mn_wifi.net import Mininet_wifi
from mn_wifi.cli import CLI
from mininet.log import setLogLevel, info
import time


def topology():
    net = Mininet_wifi()
    info("*** Creating nodes\n")
    sta1 = net.addStation('sta1', position='0,50,0')
    sta2 = net.addStation('sta2', position='5,50,0')
    net.addAccessPoint('ap1', ssid='netprog-ap', mode='g', channel='1',
                       position='0,50,0', range=30, failMode='standalone')
    info("*** Configuring wifi nodes\n")
    net.configureWifiNodes()
    info("*** Starting network\n")
    net.build()
    net.start()
    time.sleep(2)  # let both stations finish associating

    sta1.cmd('pkill -f echo_server.py')  # clean up any stale server from a previous run
    sta1.cmd('python3 echo_server.py > /tmp/echo_server.log 2>&1 &')
    time.sleep(1)
```

```
    # Part D: one benchmark near the AP
    info("*** Running benchmark at distance=5\n")
    output = sta2.cmd(f'python3 echo_bench_client.py {sta1.IP()} near')
    print(output)

    # Part E: move sta2 close to the edge of ap1's range (range=30)
    sta2.setPosition('28,50,0')
    time.sleep(1)  # let the wireless link settle after moving

    info("*** Running benchmark at distance=28\n")
    output = sta2.cmd(f'python3 echo_bench_client.py {sta1.IP()} far')
    print(output)

    CLI(net)
    sta1.cmd('pkill -f echo_server.py')
    net.stop()


if __name__ == "__main__":
    setLogLevel("info")
    topology()
```

Final `plot_rtt.py` (Part F; sheet code plus dashed average lines, averages in the legend and a y-axis starting at 0):

```
import json
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

with open("rtt_near.json") as f:
    near = json.load(f)
with open("rtt_far.json") as f:
    far = json.load(f)

avg_near = sum(near) / len(near)
avg_far = sum(far) / len(far)

plt.figure(figsize=(9, 4.5))
plt.plot(near, label=f"near (5m), avg {avg_near:.3f} ms", marker=".", color="tab:blue")
plt.plot(far, label=f"far (28m), avg {avg_far:.3f} ms", marker=".", color="tab:red")
plt.axhline(avg_near, color="tab:blue", linestyle="--", linewidth=1, alpha=0.7)
plt.axhline(avg_far, color="tab:red", linestyle="--", linewidth=1, alpha=0.7)
plt.ylim(bottom=0)
plt.grid(True, linestyle=":", alpha=0.6)
plt.xlabel("Message number")
plt.ylabel("Application RTT (ms)")
plt.title("Application-Level RTT: Near vs. Far")
plt.legend()
plt.tight_layout()
plt.savefig("rtt_plot.png", dpi=150)
print("Saved rtt_plot.png")
```

4. Lab Tasks and Results

Task 1 — Unmodified echo_server.py Inside sta1 (Part B)

```
diz@diz:~/netprog-2026/lab09_app_integration$ grep -n "HOST\|PORT\|bind" echo_server.py
4:HOST = "0.0.0.0"
5:PORT = 5000
26:    server_sock.bind((HOST, PORT))
28:    print(f"Echo server listening on {HOST}:{PORT}")
```

Figure 1: echo_server.py copied unmodified from Lab 2: HOST 0.0.0.0, PORT 5000

```
*** Starting CLI:
mininet-wifi>
mininet-wifi> sta1 python3 echo_server.py &
mininet-wifi>
mininet-wifi> sta2 python3 -c "import socket; s=socket.socket(socket.AF_INET, socket.SOCK_STREAM); s
.connect(('10.0.0.1', 5000)); s.sendall(b'hello from sta2'); print(s.recv(1024))"
b'HELLO FROM STA2'
```

Figure 2: Part B: server started on sta1, one-line client on sta2 receives the echo

The server started in the background on sta1 and the one-line client on sta2 connected to `10.0.0.1:5000` and received `b'HELLO FROM STA2'`. The reply is in capital letters, not `b'hello from sta2'` as written in the sheet, because my Lab 2 server converts each message to upper case before echoing it back — that is its own protocol, and I kept it exactly as it was. The test proves that the same server code and wire protocol work across the emulated wireless link between two separate stations.

Task 2 — Benchmark Client Summary from a Single Run (Part C)

```
mininet-wifi> sta2 python3 echo_bench_client.py 10.0.0.1 test
test: min=0.479ms avg=0.534ms max=1.154ms std=0.096ms n=50
```

Figure 3: echo_bench_client.py run once by hand from sta2 against the server on sta1

The client connected once, sent 50 messages of 64 bytes and printed **min = 0.479 ms, avg = 0.534 ms, max = 1.154 ms, std = 0.096 ms, n = 50**. The upper-case echo does not affect the timing because `b'x' * 64` comes back as `b'X' * 64` — the same 64 bytes in length.

Task 3 — Fully Automated Deployment (Part D)

```
diz@diz:~/netprog-2026/lab09_app_integration$ sudo python3 app_topo.py
*** Creating nodes
*** Configuring wifi nodes
*** ap1-wlan1: minimum tx power (1 dBm) yields 30.00m for requested 30.00m (delta +0.00m)
*** Starting network
*** Starting controller(s)
*** Starting L2 nodes
ap1 ...
*** Running benchmark at distance=5
near: min=0.486ms avg=0.537ms max=1.249ms std=0.110ms n=50

*** Running benchmark at distance=28
far: min=1.764ms avg=2.259ms max=2.531ms std=0.208ms n=50
```

Figure 4: sudo python3 app_topo.py: topology build, server launch and both benchmarks in one run

One command built the topology, killed any stale server, started `echo_server.py` on sta1 in the background and ran the benchmark from sta2, with no manual typing. The first summary line, **near: avg = 0.537 ms**, matches the manual run in Task 2 (0.534 ms), which shows the automated pipeline measures the same thing.

Task 4 — Near vs. Far (Part E)

Run	Distance	min (ms)	avg (ms)	max (ms)	std (ms)	n
test (Task 2)	5 m	0.479	0.534	1.154	0.096	50
near	5 m	0.486	**0.537**	1.249	0.110	50
far	**28 m**	**1.764**	**2.259**	**2.531**	**0.208**	50
far / near		3.6×	**4.2×**	2.0×	**1.9×**	

```
diz@diz:~/netprog-2026/lab09_app_integration$ sudo chown $USER:$USER rtt_near.json rtt_far.json
diz@diz:~/netprog-2026/lab09_app_integration$ ls -l rtt_*.json
-rw-r--r-- 1 diz diz  982 Oct 29 15:37 rtt_far.json
-rw-r--r-- 1 diz diz 1004 Oct 29 15:37 rtt_near.json
```

Figure 5: Both result files produced by the run: rtt_near.json and rtt_far.json

Both runs completed (Figure 4). At 28 m the average RTT was **2.259 ms, about 4.2× the near value of 0.537 ms**, and even the fastest far message (1.764 ms) was slower than every near message except the first. The far run is also more variable: its standard deviation roughly doubled (0.110 → 0.208 ms). The far average is a little above the 1.87 ms predicted from Lab 8's ping, which is expected because the application RTT also includes the TCP stack and the Python interpreter on both ends.

Task 5 — RTT Plot (Part F)

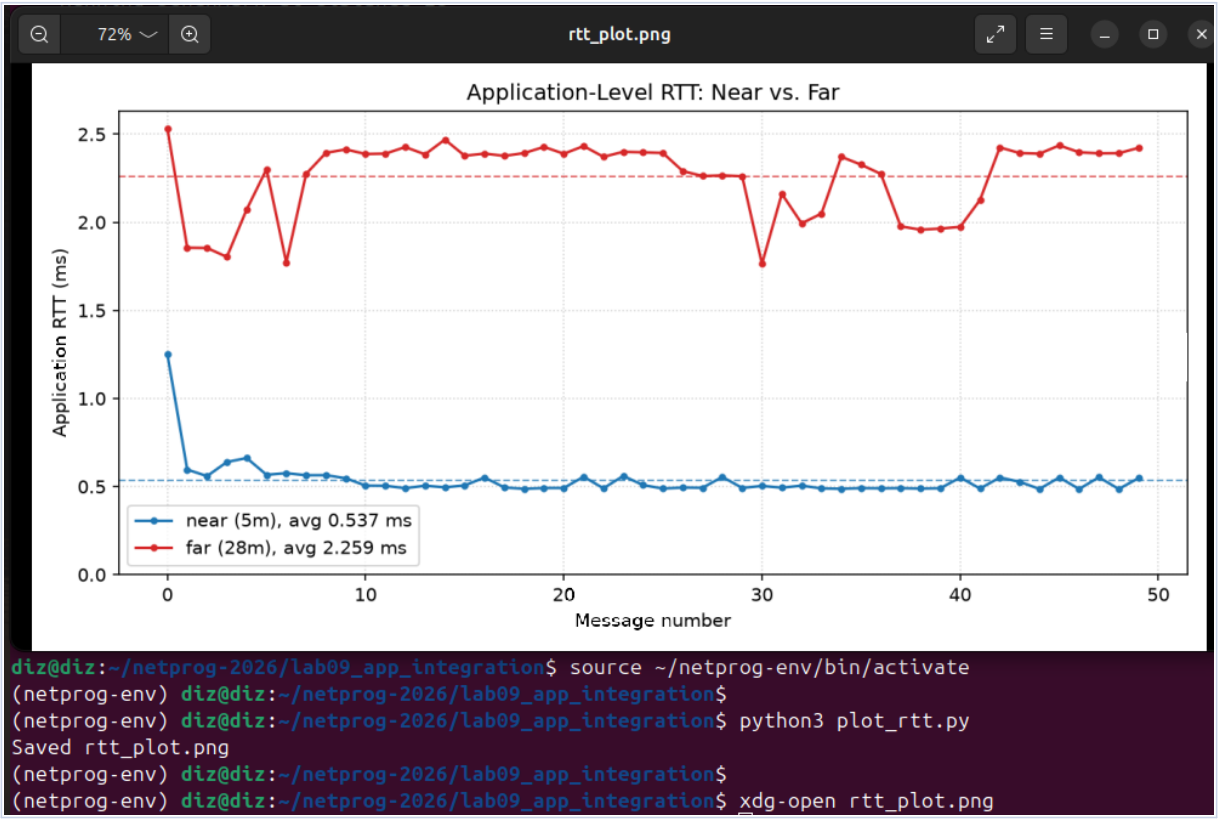


Figure 6: rtt_plot.png: per-message application RTT, near (5 m) vs. far (28 m); dashed lines are the averages

The two series are completely separated. The **near** series (blue) starts with one higher value of 1.25 ms for the first message — a warm-up cost (caches and the first exchange on the new connection) — and then stays flat between about 0.48 and 0.66 ms. The **far** series (red) sits around 2.3–2.5 ms for most messages and shows several dips to about 1.8–2.0 ms (messages 1–6, around 30–33 and 37–40), so it wanders over a much wider band of ≈0.8 ms compared with ≈0.2 ms for the near series after warm-up. The far line is therefore both higher and visibly less stable.

Task 6 — Zero Code Changes, and Comparison with Lab 8

`echo_server.py` ran with zero changes because every Mininet-WiFi station is a real Linux network namespace with its own interface, IP address and kernel TCP/IP stack: the program calls the same `socket()`, `bind()` and `accept()` as on a laptop. This is the emulation-vs-simulation theme from Lab 1 — an emulator runs the real software over a modelled network, while a simulator would need the application to be rewritten as a model. Compared with Lab 8, this lab measures a two-way request/response latency of a real application, whereas iperf3 measured one-way UDP throughput, jitter and loss: at 25–30 m Lab 8 showed 0 % loss and only a 20 % throughput drop, but here the latency effect of distance is much larger (4.2× higher RTT).

5. Verification Checklist

#	Checkpoint	Evidence	✓
1	Unmodified echo_server.py runs inside sta1 and responds correctly (Part B) — reply is upper case by design of the Lab 2 server	Figures 1–2	✓
2	echo_bench_client.py connects once and correctly times count = 50 messages (Part C)	Figure 3	✓
3	app_topo.py automates server launch + one benchmark run end-to-end (Part D)	Figure 4	✓
4	Near and far benchmark runs both complete and produce rtt_near.json / rtt_far.json (Part E)	Figures 4–5	✓
5	rtt_plot.png shows both series clearly labeled and distinguishable (Part F)	Figure 6	✓
6	Far-distance RTT is visibly higher/more variable than near (avg 2.259 vs 0.537 ms, std 0.208 vs 0.110 ms)	Task 4 table	✓

6. Reflection Questions

Q1. Explain why echo_server.py from Lab 2 could be deployed inside a Mininet-WiFi station without any code modification. What does this tell you about the relationship between emulation and ordinary application code?

A station is not a model of a host but a real Linux network namespace with its own wlan interface, IP address (10.0.0.1) and the normal kernel TCP/IP stack, so the server's `bind(('0.0.0.0', 5000))` and `accept()` behave exactly as on a physical machine — only the wireless link underneath is emulated. Emulation therefore lets ordinary, unmodified application code be tested over a controllable network: the same file I wrote in Lab 2 could be deployed on real hardware afterwards without changes, which a pure simulator cannot offer.

Q2. Why was a new, non-interactive benchmark client needed instead of directly reusing Lab 2's interactive client.py for this lab's automated measurements?

Lab 2's client waits for `input()` from a person at a keyboard, but `sta2.cmd()` runs a command unattended and blocks until it finishes, so the interactive client would wait forever and the script would hang. A measurement also has to be repeatable: the benchmark client sends a fixed 50 messages of a fixed 64 bytes, times each one with `perf_counter()`, prints one summary line and saves the raw RTTs to JSON, so the near and far runs are directly comparable and can be plotted automatically.

Q3. Compare your application-level RTT measurements (Part E) to Lab 8's iperf3-based measurements at similar distances. What might explain any differences you observed?

Distance	Lab 8 (iperf3 UDP / ping)	Lab 9 (application RTT)
≈5 m	10.00 Mbps delivered, jitter 0.24 ms, loss 0 %, ping RTT 0.69 ms	avg 0.537 ms, std 0.110 ms
≈28–30 m	7.93 Mbps delivered (30 m), jitter 0.47 ms, loss 0 %, ping RTT 1.97 ms	avg 2.259 ms (28 m), std 0.208 ms

Both methods show the same direction — performance degrades with distance and nothing is lost — but they measure different things. iperf3 sends a continuous one-way UDP stream and reports throughput, jitter and loss, so distance appeared only as a 20 % throughput drop; my client sends one small message at a time and waits for the reply, so it measures latency directly and shows a 4.2× increase. The application RTT at 28 m (2.26 ms) is also higher than Lab 8's ping at 30 m (1.97 ms) because it includes TCP processing and two Python processes on top of the kernel path that ICMP ping uses, while at 5 m it was slightly lower (0.54 vs 0.69 ms) because 50 back-to-back messages keep the path warm, whereas ping sent only three packets one second apart.

7. Conclusion

My Lab 2 `echo_server.py` ran inside a Mininet-WiFi station with zero code changes, confirming that stations are real Linux hosts and that emulation runs ordinary application code. A non-interactive benchmark client and one automation script were enough to deploy the server, run both benchmarks and save the results. Moving sta2 from 5 m to 28 m raised the average application RTT from 0.537 ms to 2.259 ms (4.2×) and roughly doubled its variability (std 0.110 → 0.208 ms), consistent with the distance-dependent delay already seen with iperf3 and ping in Lab 8.